



**COMSATS University Islamabad,
Abbottabad Campus**

SOFTWARE DESIGN DESCRIPTION

(SDD DOCUMENT)

for

SmartCrop – AI & IoT-Based Crop Suitability and Monitoring System

Version 1.0

By

Aleeza Javed

CIIT/SP23-BCS-141/ATD

Zulkifl Ahmad

CIIT/SP23-BCS-047/ATD

Muneeba Kalsoom

CIIT/SP23-BCS-147/ATD

Supervisor

Sir Yasar Khan

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

1. Introduction	1
2. Design methodology and software process model	1
3. System overview	1
3.1 Architectural Design	2
3.2 Process Flow / Representation	5
4. Design Models	7
4.1 System Sequence Diagram (SSD)	7
4.2 Class Diagram	8
4.3 Sequence Diagram	9
4.5 Package Diagram	11
4.6 Data Flow Diagram	11
4.7 JSON Tree	13
4.8 System Block Diagram	15
4.9 Hardware Schematic Diagram	16
4.10 Timing Diagram	17
5. Data Design	18
5.1 Data Dictionary	18
6. Algorithm & Implementation	21
6.1 Algorithm 1 – User Authentication	22
6.2 Algorithm 2 – Sensor Data Acquisition	22
6.3 Algorithm 3 – Crop Recommendation	23
6.4 Algorithm 4 – Yield Prediction	23
6.5 Algorithm 5 – Fertilizer Recommendation	24
6.6 Algorithm 6 – Weather Retrieval	24
6.7 Algorithm 7 – Irrigation Control	24
7. Software Requirements Traceability Matrix	25
8. Human Interface Design	26
8.1 Screen Images	27
8.2 Screen Objects and Actions	32
9. References	34

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Sir Yasar Khan

Signature_____

1. Introduction

SmartCrop – AI & IoT-Based Crop Suitability and Monitoring System is a smart farming system which combines the Internet of Things (IoT), Artificial Intelligence (AI), Cloud Computing and mobile technologies to enable intelligent decision making in agriculture. The system developed here aims to measure the real-time soil and environmental parameters, analyze the data collected and make decisions for the farmers on different aspects of farming such as crop selection, fertilizer application, crop yield estimation, decision on irrigation scheduling based on weather forecast and remote irrigation control using a smartphone application.

The Software Design[5] Description (SDD) presents the overall software design and implementation of the SmartCrop system. It describes the system architecture, software modules, data organization, process flow, design models, algorithms, and user interface. The SmartCrop system consists of modules for user authentication and profile management, real-time sensor monitoring, crop recommendation, yield prediction, fertilizer recommendation, weather forecasting, smart irrigation control, Firebase cloud services, and ESP32-based IoT data acquisition. These modules work together to monitor field conditions, provide AI-based recommendations, store historical data, and enable remote irrigation management through the Flutter mobile application.

2. Design methodology and software process model

The system 'SmartCrop' has been designed based upon the methodology Object Oriented Design (OOD)[7] and Incremental Software Process Model. This will enable a modular, maintainable, and scalable software architecture, which will facilitate the integration of mobile application development, cloud computing, IoT devices, and machine learning models.

It was chosen because the system has multiple independent but interrelated modules like user authentication, sensor monitoring, crop recommendation, yield prediction, fertilizer recommendation, weather forecasting and irrigation control. Each module can be designed as a separate class or a service with well-defined duties and responsibilities which can be executed in a compact manner with low coupling, high cohesion, high code reusability and easy maintainability. This project employs the main development paradigm used by the project, which is object-oriented programming, so it is suitable to implement the system using OOD.

The Incremental Process Model was adopted for the development of SmartCrop, allowing the system to be built and tested in multiple stages while continuously integrating new features. The project progressed from planning, requirement analysis, and documentation to the development of the Flutter application[1], ESP32[3] sensor integration, Firebase services[2], AI-based crop, yield, and fertilizer recommendation modules, followed by weather forecast integration, smart irrigation

control, system testing, optimization, and deployment. This modular and object-oriented approach makes the SmartCrop system flexible and scalable, enabling future enhancements such as pest and disease detection, additional IoT sensors, multilingual support, and web-based farm management.

3. System overview

SmartCrop is an integrated smart agriculture system, the fusion of the Internet of Things (IoT), Artificial Intelligence (AI), cloud computing and mobile technologies, to support the intelligent decision for farmers. Multiple sensors are connected to an ESP32 microcontroller within the agriculture field, which constantly records the field's conditions. The sensor data gathered is then sent via Wi-Fi to Firebase Realtime Database, which securely stores and synchronizes the data to the mobile app.

The SmartCrop mobile application, developed using Flutter, serves as the primary interface for farmers to monitor real-time sensor data, view historical records, receive AI-based recommendations, and control irrigation remotely. The system consists of modules for user authentication, real-time sensor monitoring, crop recommendation, yield prediction, fertilizer recommendation, weather forecasting[4], smart irrigation control, Firebase cloud services, and ESP32-based IoT monitoring. This modular architecture ensures efficient communication between system components while improving scalability, maintainability, and overall system performance.

3.1 Architectural Design

The SmartCrop system operates with a Figure 3.1 Architecture that separates process/service roles into independent layers and connects them via secure cloud services. The architecture offers better maintainability, scalability, and modularity, in addition to facilitating seamless interaction between the mobile app, cloud systems, machine learning services, and IoT devices.

The architecture consists of the following layers:

3.1.1 Presentation Layer

The Presentation Layer includes the Flutter mobile application which gives a user friendly interface for farmers to Register, Sign in, monitor sensor values, get smart tips from the System, and view past readings, check weather forecast and control irrigation remotely.

3.1.2 Cloud Service Layer

Firebase offers cloud-based[9] services such as user authentication, real-time database synchronisation and secure storage of user information, sensor data, irrigation status as well as

historical agriculture data. It serves as the communication hub between mobile app and IoT hardware device.

3.1.3 Intelligence Layer

The machine learning models and weather forecasting services are stored in this layer. Data from the sensors is used to derive the crop recommendations, yield prediction and fertilizer recommendations obtained from Firebase. The Weather API provides weather data and forecasts for irrigation planning.

3.1.4 IoT Layer

The IoT[10] layer is a combination of the ESP32[3] microcontroller, which is directly connected to soil moisture, temperature, humidity, pH, NPK and Electrical Conductivity (EC) sensors. The ESP32 is continuously gathering environmental data and uploading it to Firebase via Wi-Fi. Also, it is programmed with the commands of irrigation control from Firebase to control the relay module connected to the water pump.

3.1.5 Hardware Layer

All the agricultural sensors, irrigation pump, power supply, and relay modules installed in the field are part of the Hardware Layer. These devices are sensitive to their surroundings and gather data and make irrigation decisions.

The layered structure provides a separation between the various layers, thus each one has a specific function and is independent of the other components, facilitating maintenance, testing and system expansion in the future.

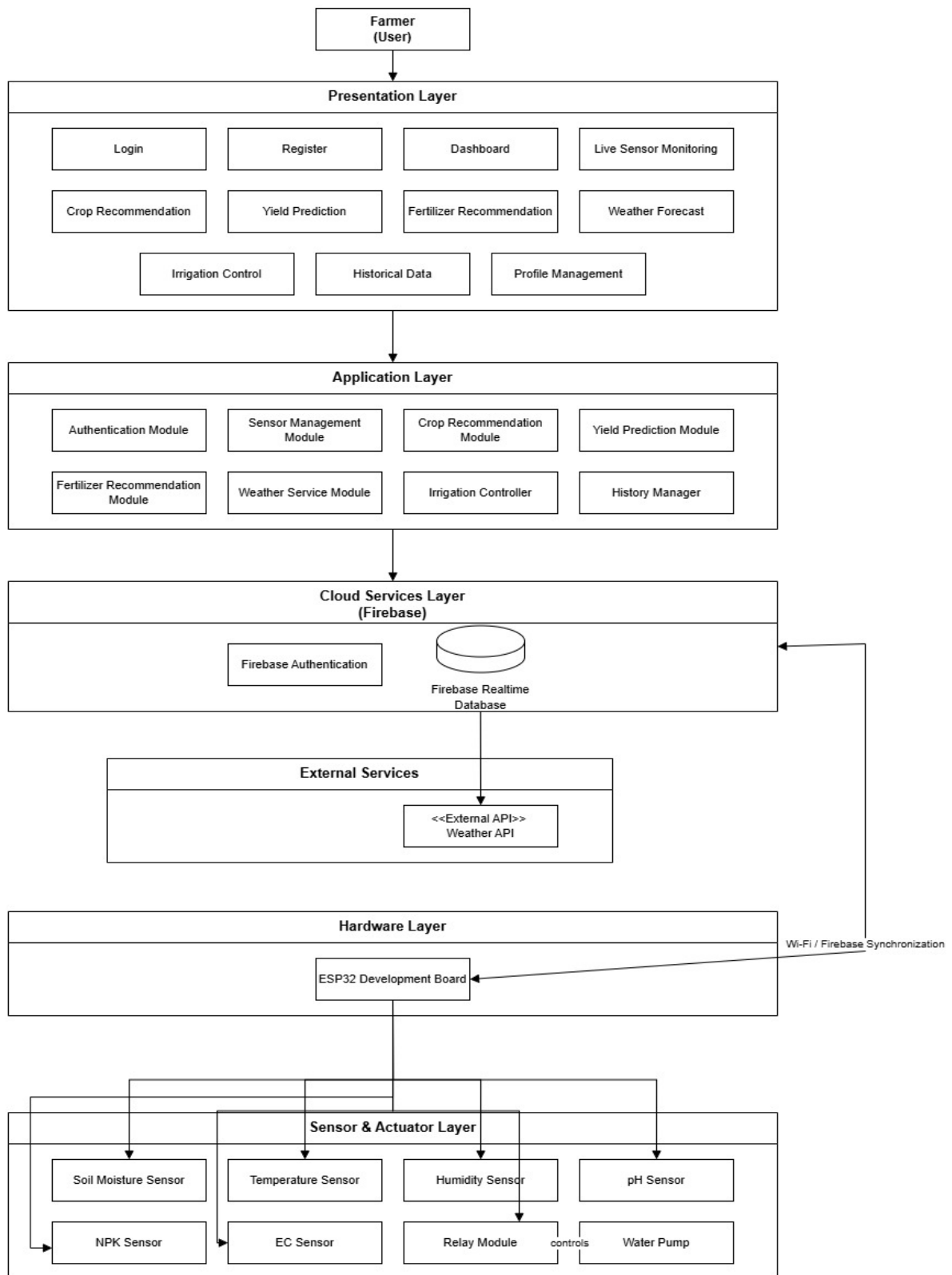


Figure 3.1 Layered Architecture Diagram

3.2 Process Flow / Representation

The essence of the SmartCrop system is a series of interlinked processes starting from collecting environmental data and ending with the intelligent support for the farmer with decisions. It begins with the ESP32 taking soil and environmental readings from the connected sensors. These readouts are sent to a Firebase Realtime Database via Wi-Fi, which synchronizes the readouts with the Flutter mobile application.

The application then fetches the most recent sensor readings from Firebase and passes the corresponding ones to the machine learning models when they are accessed from the dashboard. The system makes crop recommendations, crop yield prediction, and recommends suitable fertilizer based on the processed information, considering soil nutrient levels. At the same time, the weather forecast information is fetched from the Weather API to support farmers in irrigation decision making.

The farmer presses the "Start irrigation" button on the mobile app, which is saved in Firebase. The ESP32 keeps checking the irrigation status in Firebase, and when it detects it is on, it turns on the relay module and water pump, and vice versa, so that the water pump can be controlled remotely. In addition, historical sensor readings and recommendations are also saved in the database for future monitoring and analysis.

The entire flow of activities Figure 3.2 enables constant communication between the IoT system and the mobile app, artificial intelligence, and farm services, all of which contribute to effective and data-driven farming methods.

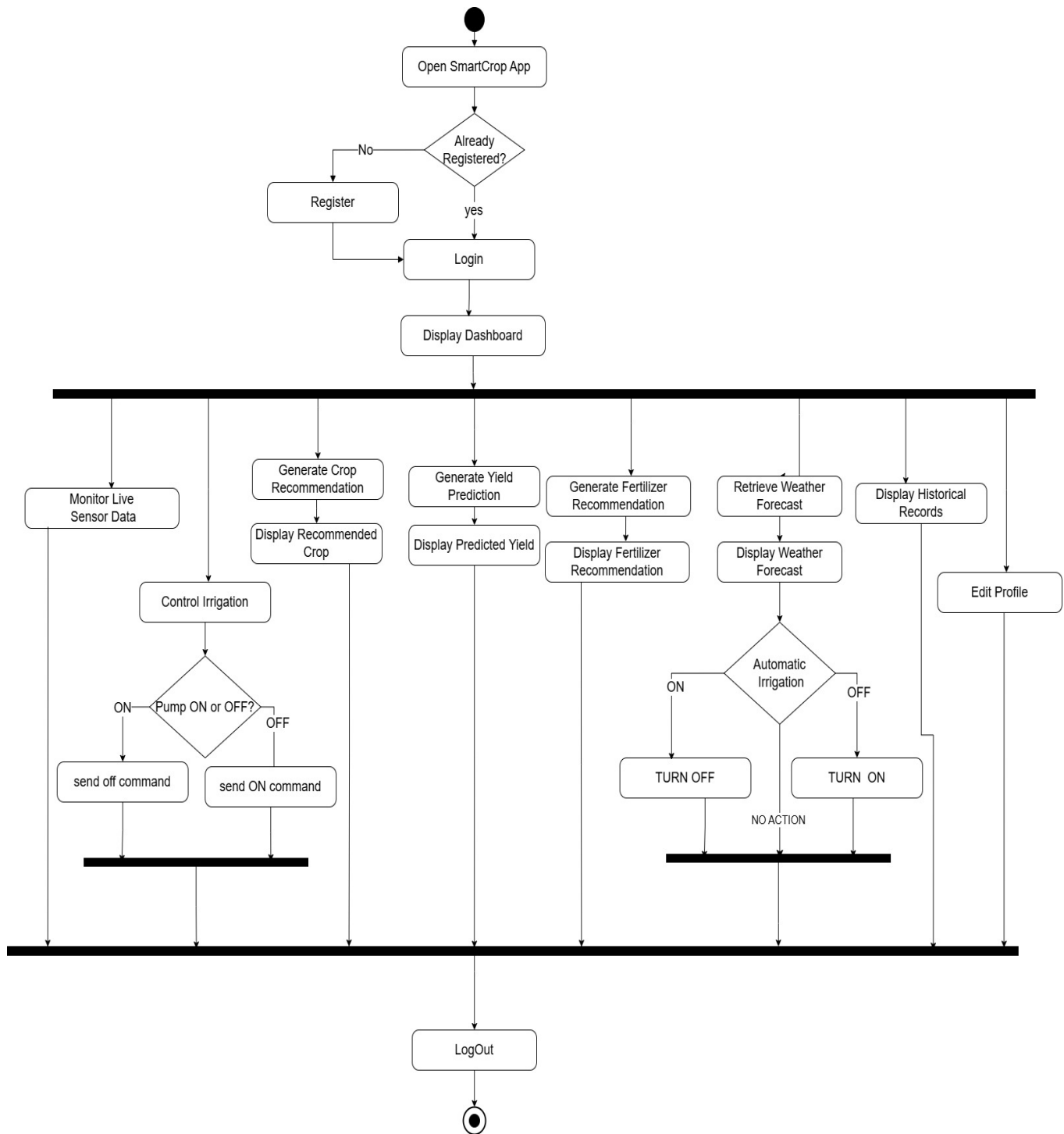


Figure 3.2 Activity Diagram

4. Design Models

The SmartCrop system is designed based on Object Oriented Design (OOD). Various models have been created like UML and system design models, to represent various aspects of the system. These models show the structural, behavioral, functional and hardware views of the system. They together give the deep information of interaction between various software components, hardware modules and external services to attain desired functionality. The following sections outline all the design models that were employed in the development of SmartCrop.

4.1 System Sequence Diagram (SSD)

The System Sequence Diagram (SSD) Figure 4.1 depicts the interaction between the major actor/plant owner (Farmer) and the SmartCrop system in a typical farming session. Unlike a detailed sequence diagram, the SSD scope is on the entire use of SmartCrop application and only considers interactions with the external components (or services) of the application (in this terms, the external components are the user).

The farmer first registers to the application. Once authenticated the farmer logs in to the dashboard, the farmer will see the latest data that has been recorded by the sensors. The farmer can then ask for a recommendation of the crop, predicted yield, fertilizer recommendations, as well as the weather information. When it is needed, the farmer sends a command to irrigation which is interpreted by the system and transmitted to the ESP32 via Firebase. The farmer can also update his/her profile information or sign out from the application.

The following SD, which is detailed, is based on the SSD, which gives a high level view of how the system works.

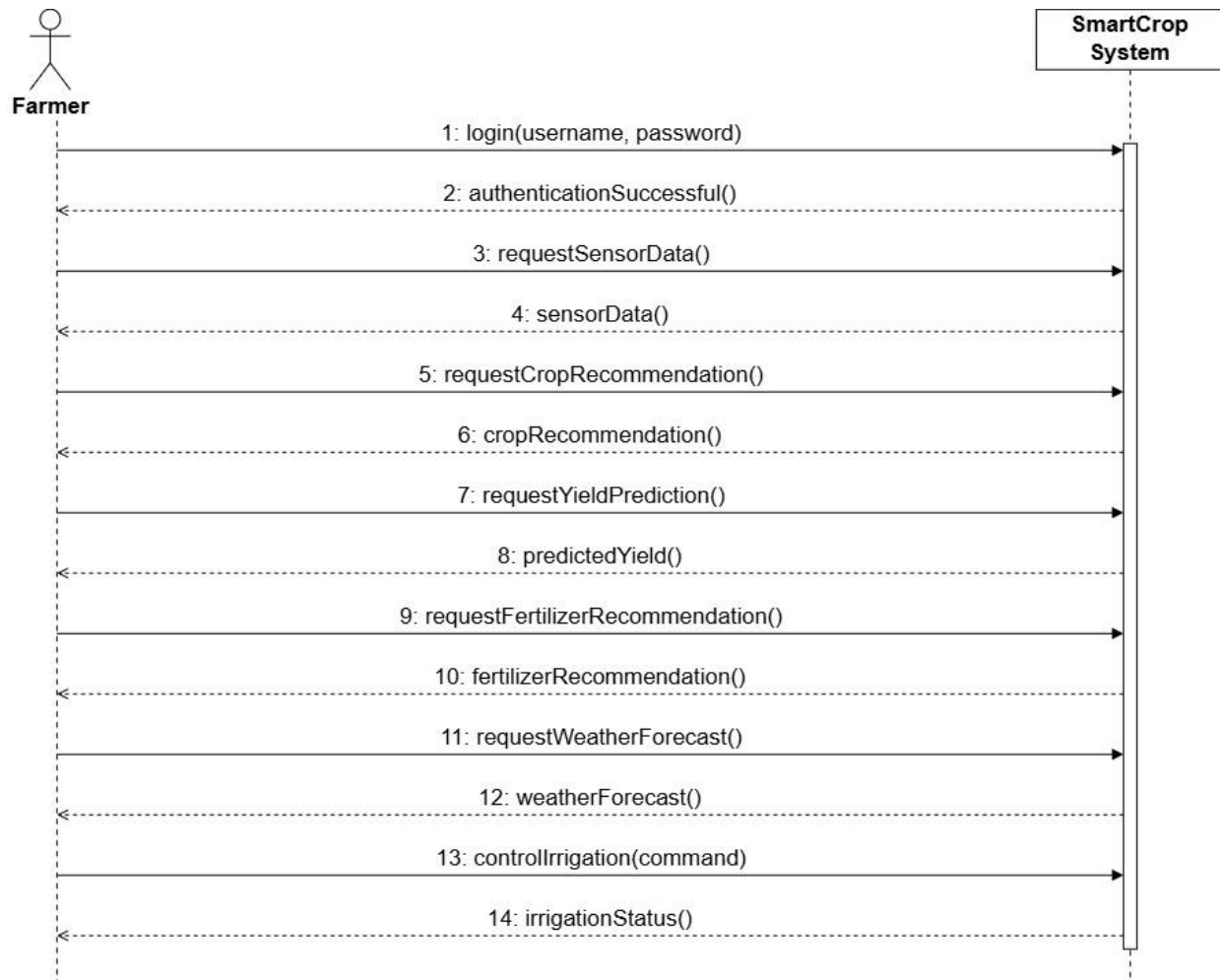


Figure 4.1 System Sequence Diagram

4.2 Class Diagram

Figure 4.2 shows the static structure of the SmartCrop system by showing the software classes, attributes, operations and relationships. It is an object-oriented architecture in which, each module is defined as a class that performs a specific function.

The main classes are User, SensorData, CropRecommendation, YieldPrediction, FertilizerRecommendation, WeatherData, Irrigation. In these, they work together to manage user authentication, to process sensor readings, to make AI recommendations, to fetch weather data and to control irrigation operations.

Association, dependency and aggregation relationships (as appropriate) are represented between classes. This modularity approach makes the code reusable, maintainable, scalable, and clearly separates responsibilities.

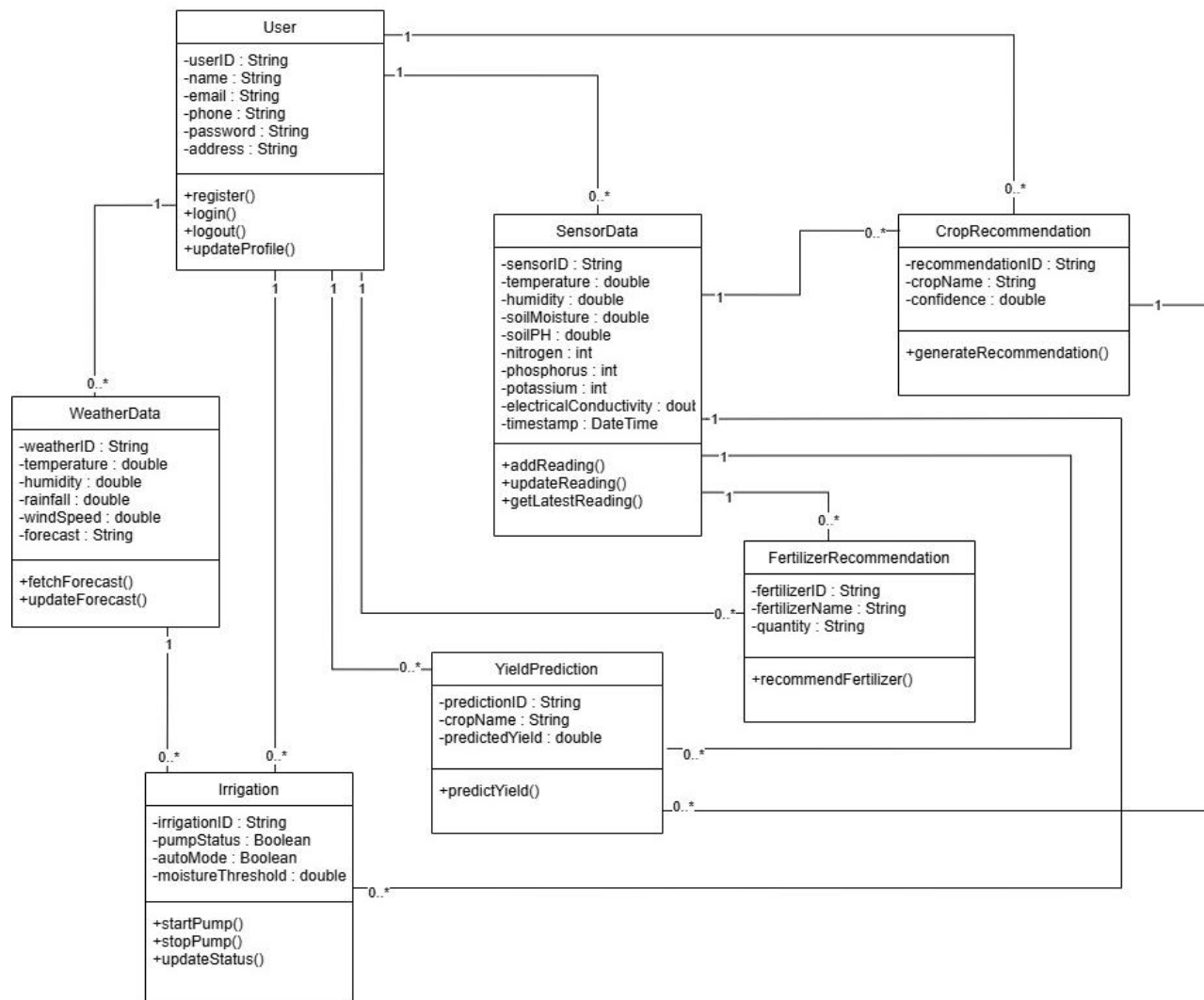


Figure 4.2 Class Diagram

4.3 Sequence Diagram

Figure 4.3 is used to show the interactions between objects at various stages of the crop recommendation process. This model shows the communication between each software component such as the Flutter app, Firebase services, machine learning modules, weather service, and the ESP32 controller, unlike the System Sequence Diagram.

The farmer sends a request for recommendations via the dashboard and the mobile app fetches sensor data from Firebase and passes on the requested data to the machine learning module. The model receives crop data, yields a crop recommendations, yield prediction and fertilizer recommendation. At the same time, the Weather API is used to retrieve the weather and show it in

the application. When the farmer starts the irrigation, the command is sent to Firebase and returned by the ESP32 which is connected to the relay module that controls the water pump.

In the sequence diagram, the sequence of the messages exchanged between the system's components during normal system functioning is illustrated.

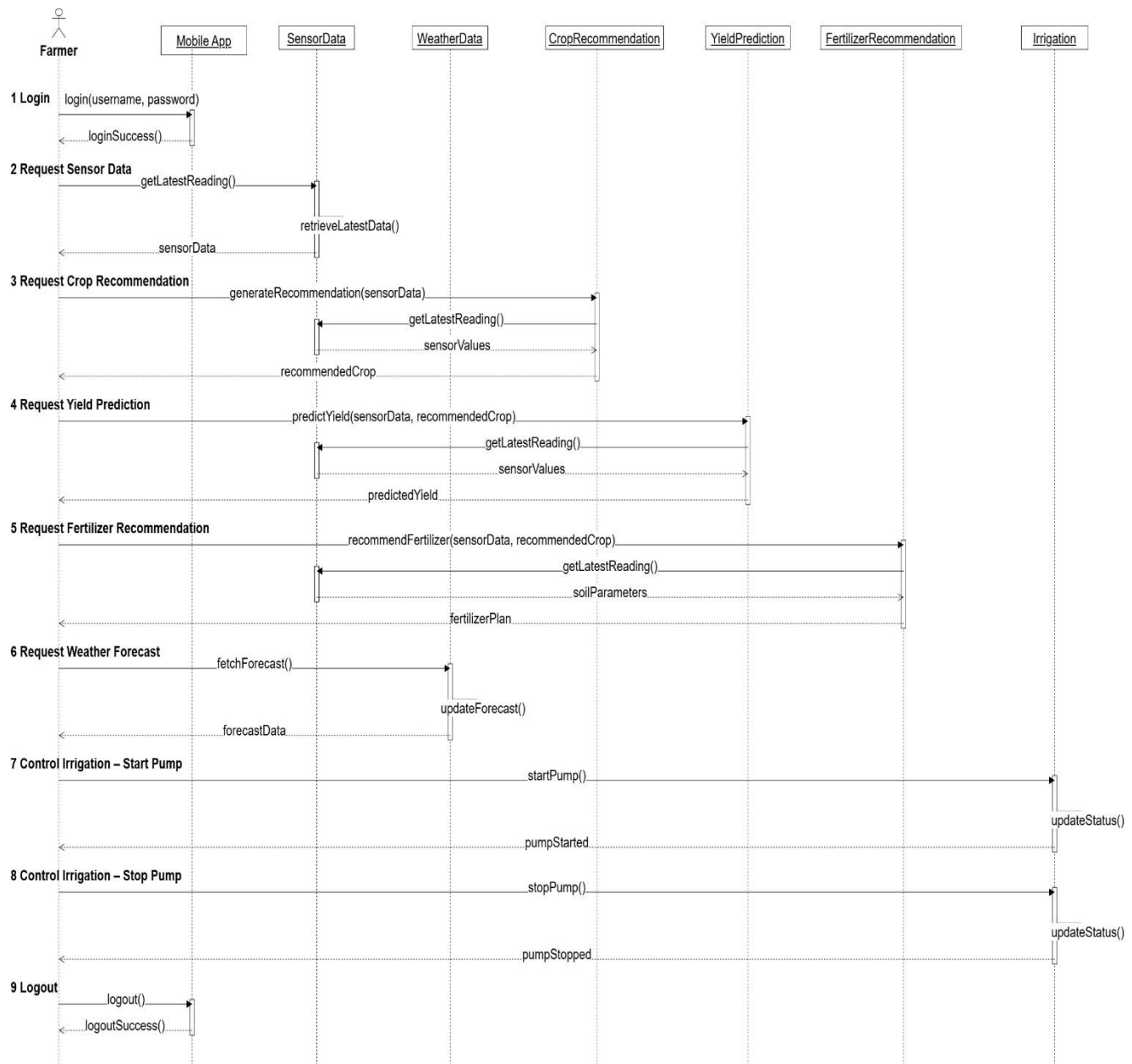


Figure 4.3 Sequence Diagram

4.5 Package Diagram

The logical division of the SmartCrop software into logical independent functional packages is shown on the Figure 4.4. There are packages provided with the application such as Authentication, Dashboard, Recommendation, Weather, Irrigation, Firebase Services, Machine Learning, and Hardware Communication.

A package organizes together related classes and services that share a common responsibility. This module structure will improve ease of adding features in the future as well as testing, debugging, and software development because of the reduction of dependency on the other system elements.

The package structure also makes the application more maintainable because, if one part of the application changes, it has minimal effects on other parts of the application.

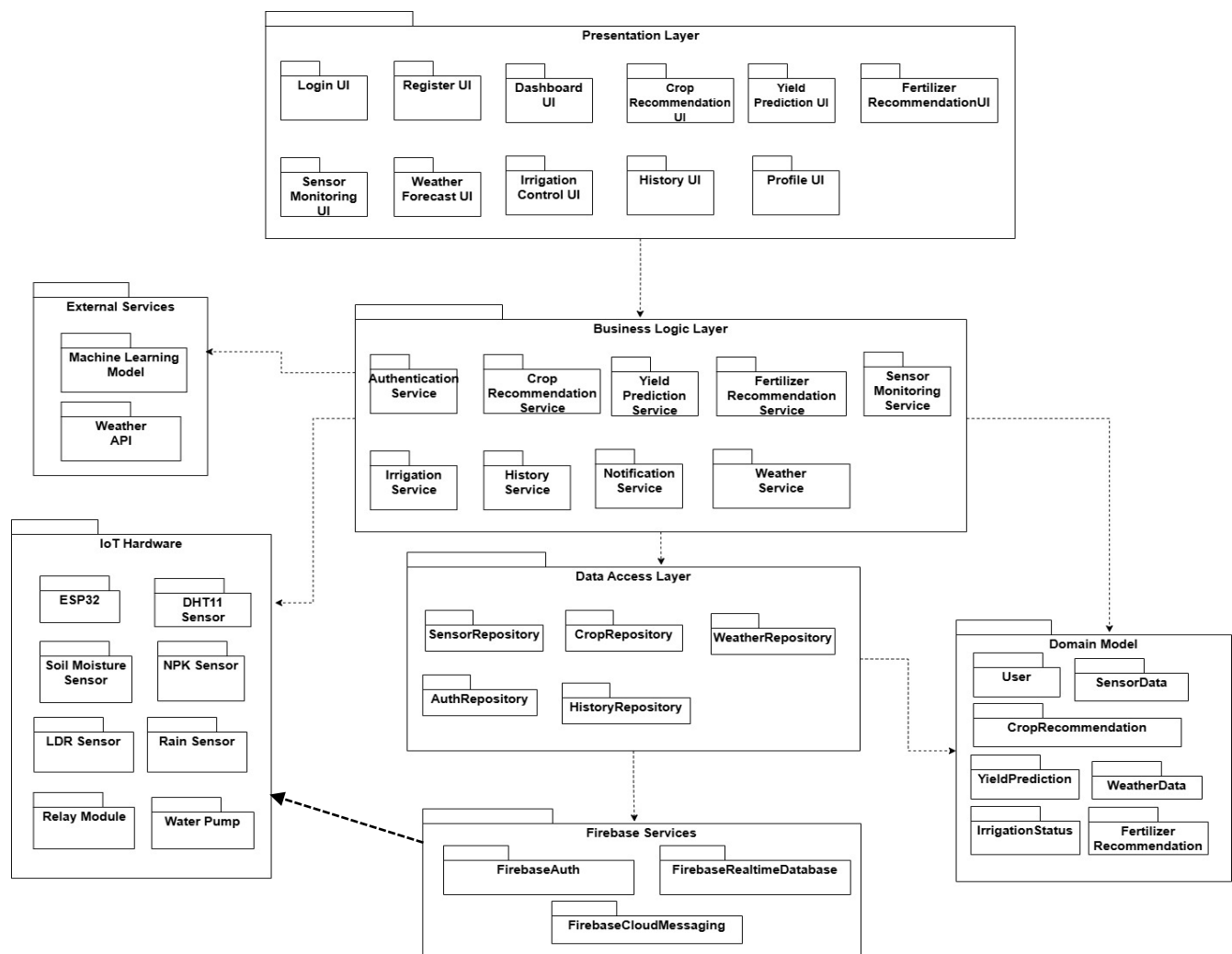


Figure 4.5 Package Diagram

4.6 Data Flow Diagram

Figure 4.5 shows the overall SmartCrop system as a single, high-level process communicating with external systems. The main external entity is the Farmer who supplies authentication data, and who requests agricultural services and gets recommendations and monitoring data from the system. The irrigation commands given to the SmartCrop system are sent back to the ESP32 to be executed; likewise, sensor data collected on the ESP32 is supplied to the SmartCrop system.

The Level 0 DFD gives a high-level view of the flow of data into and out of the system, while hiding the detail of the processing within the system.

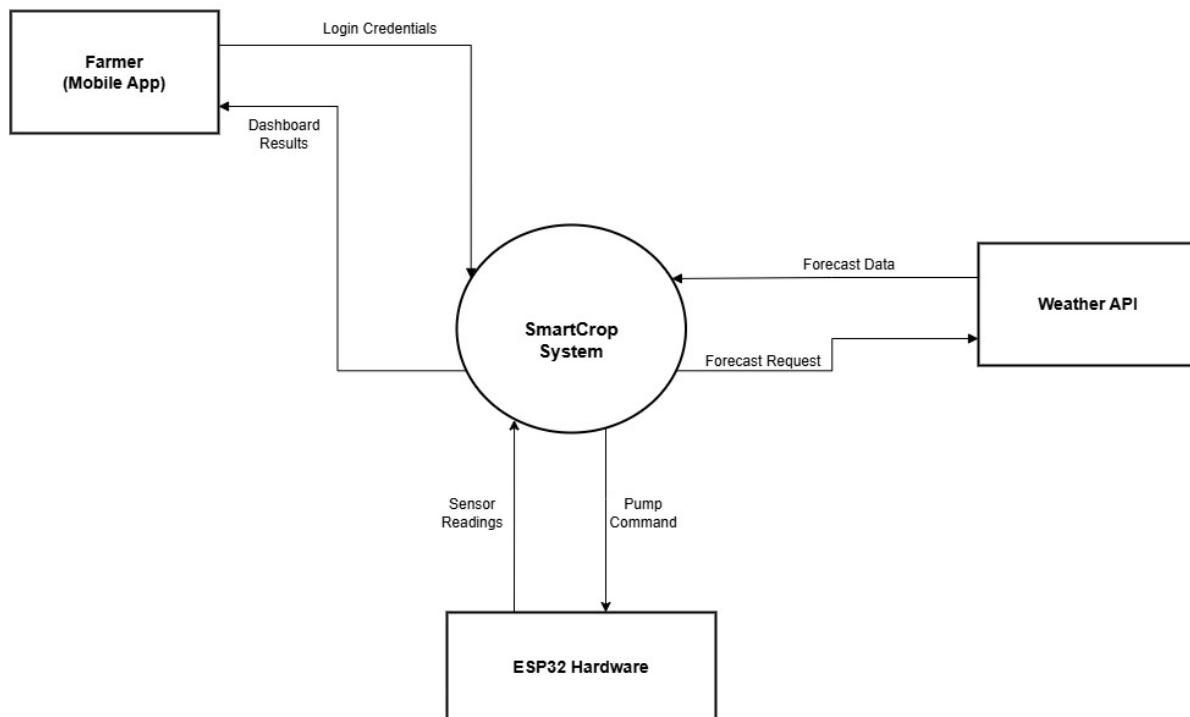


Figure 4.5 DFD Level 0

Figure 4.6 is a Level 1 view of the SmartCrop system that breaks down the major functional processes of the system. They comprise of User Authentication, Dashboard Management, Sensor Monitoring, Crop Recommendation, Yield Prediction, Fertilizer Recommendation, Weather Forecasting, Irrigation Control and Profile Management.

Individual processes send and receive data off Firebase Realtime Database and relay back to the farmer via the cell phone app. This level gives a better insight into the information flow among the different primary software modules.

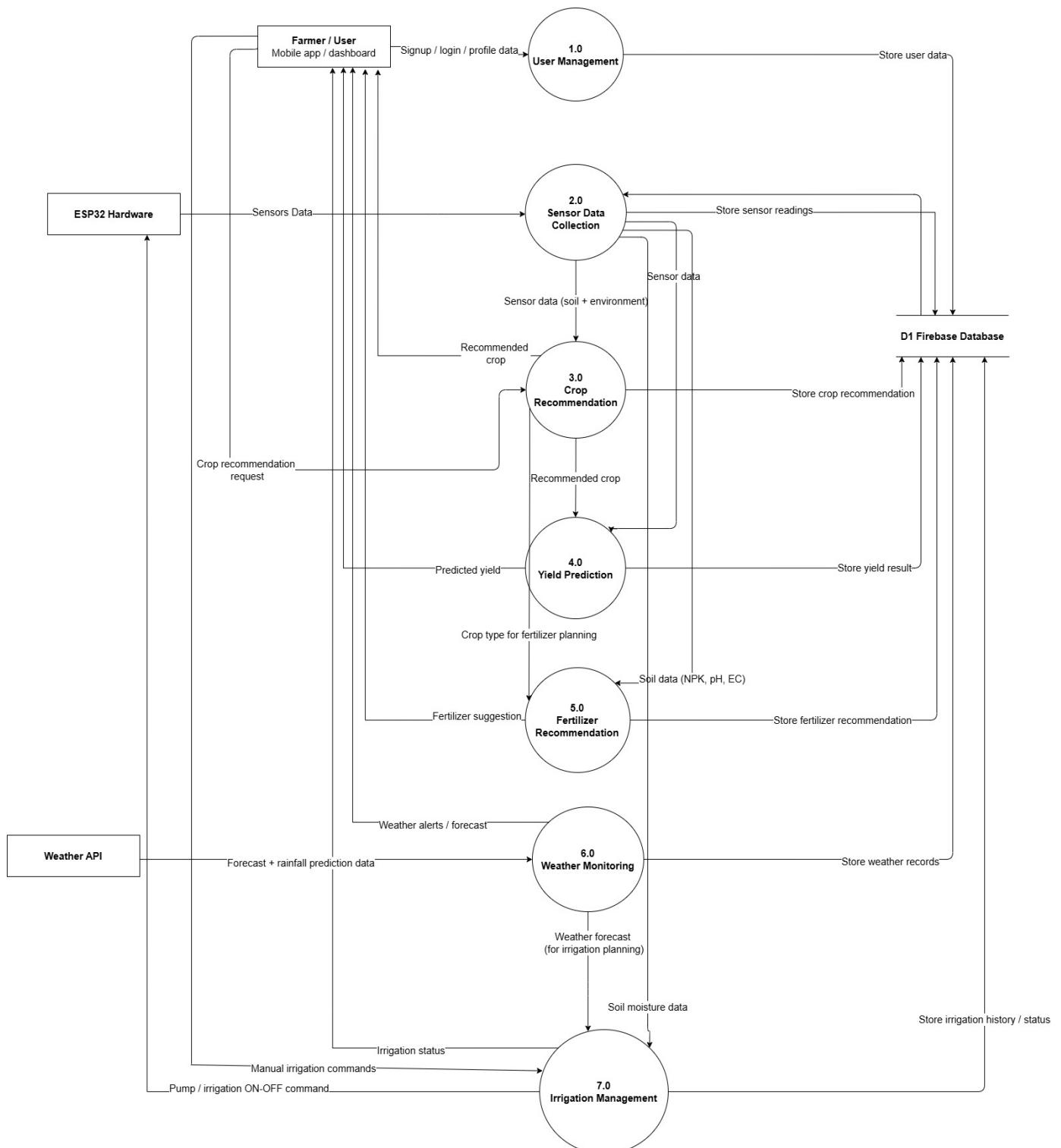


Figure 4.6 DFD Level 1

4.7 JSON Tree

Figure 4.7 is used to describe the hierarchical structure of data in the Firebase Realtime Database. It shows the data organization that happens in the cloud data base. It displays how the information of the user, the sensor readings, the result from the machine learning, the weather data, and the status of the irrigation are structured in the cloud data base. Since Firebase is a NoSQL database, data is stored as nested JSON objects instead of relational tables.

The JSON format allows for seamless real-time updates and synchronization of data between the ESP32 microcontroller, Firebase cloud services, and the Flutter mobile application. Personal information is stored in the unique identifier (UID) of each user and sensor values, results of recommendation, weather information, and status of irrigation are stored in separate nodes. This can provide a quick way to get data when needed, make updating easy, and enable future improvements when necessary.

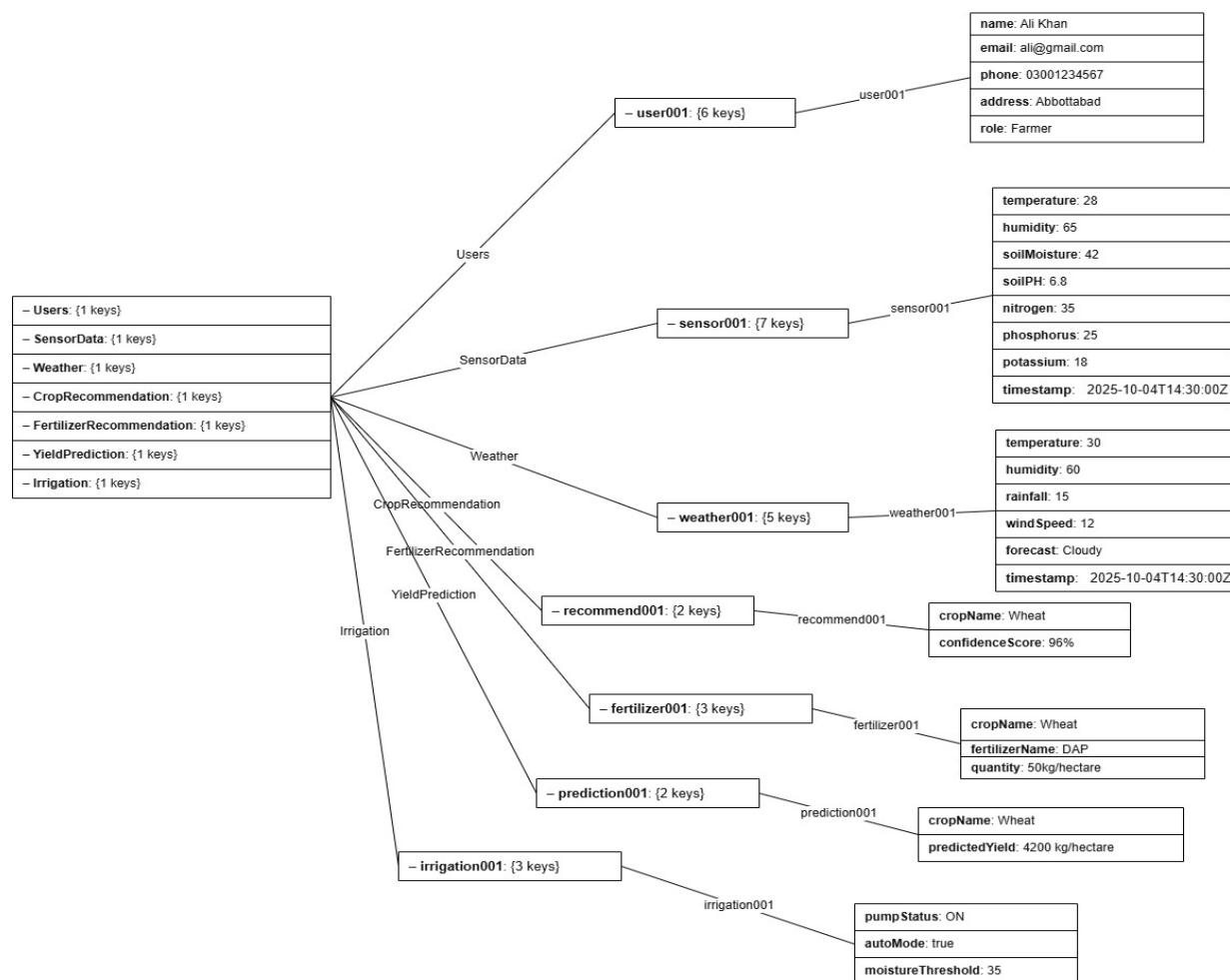


Figure 4.7 Firebase JSON Tree

4.8 System Block Diagram

Figure 4.8 offers a high level view of the physical hardware components of the SmartCrop system and their connections to each other. It demonstrates the wiring of various environmental sensors with the central processing unit (ESP32 microcontroller) for the acquisition of data from the sensors and for control of the devices.

The hardware setup includes soil moisture sensor, soil temperature sensor, humidity sensor, pH sensor, NPK sensor and Electrical Conductivity (EC) sensor which are all connected to the ESP32. The microcontroller will continuously read in the sensor data and transmit to the relay module which controls the irrigation water pump. The diagram is just a high-level look at the hardware architecture and not cloud services or software components.

Modular hardware design enables easy expansion of the system with the addition of further sensors or actuators to the system without extensive hardware changes.

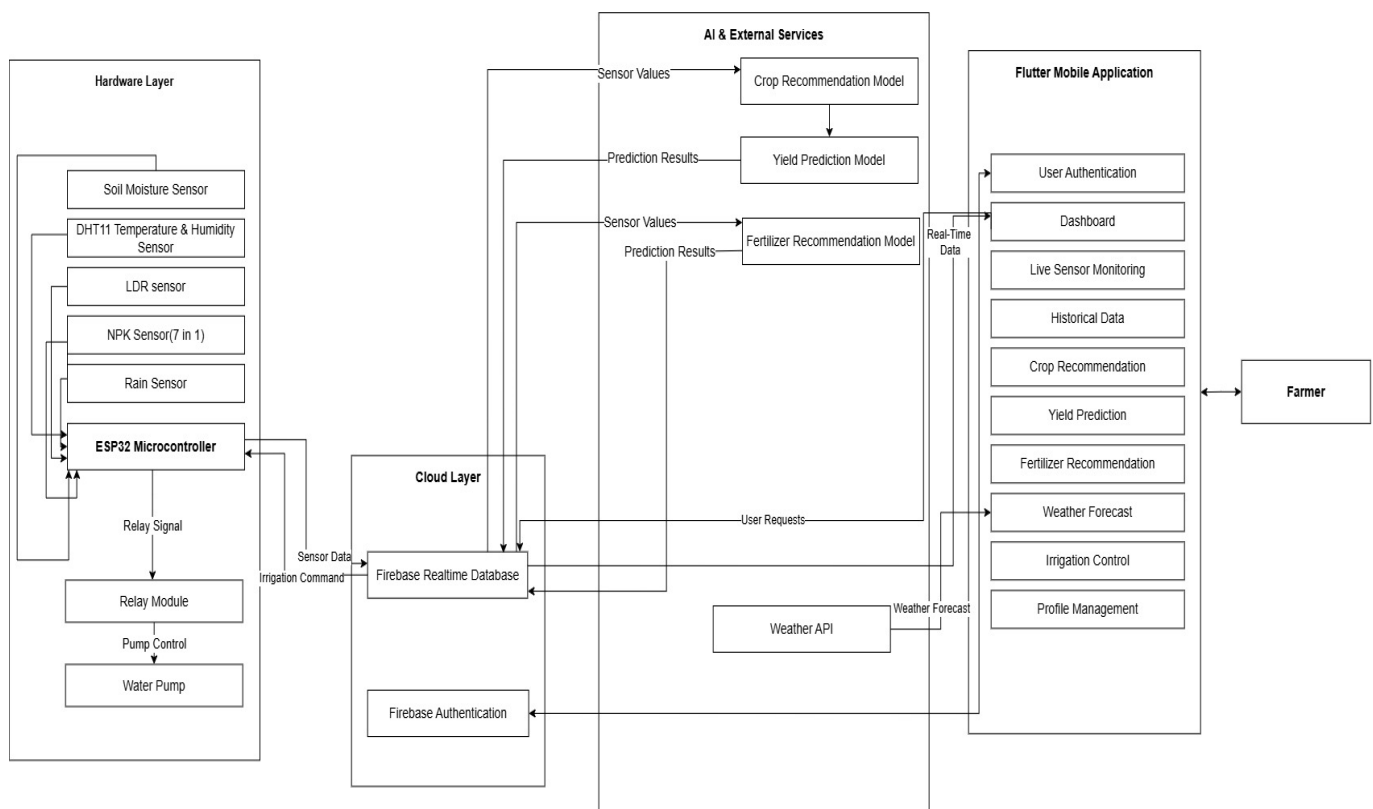


Figure 4.8 System Block Diagram

4.9 Hardware Schematic Diagram

The Hardware Schematic Diagram shows the wiring connections of ESP32 microcontroller, sensors, relay module, and irrigation pump. The schematic differs from the Hardware Block Diagram which just supplies a conceptual view, by clarifying the actual wiring necessary to implement the hardware system.

All the sensors are mapped to the respective GPIO or communication port of the ESP32 which supports them. The relay module is interfaced with a digital output pin of the ESP32 to control the water pump. The connections for the power supply and earth ground are also provided so all the hardware works properly.

The schematic is used to reference in the assembly of the hardware and during testing, troubleshooting, and maintenance of the system in the future.

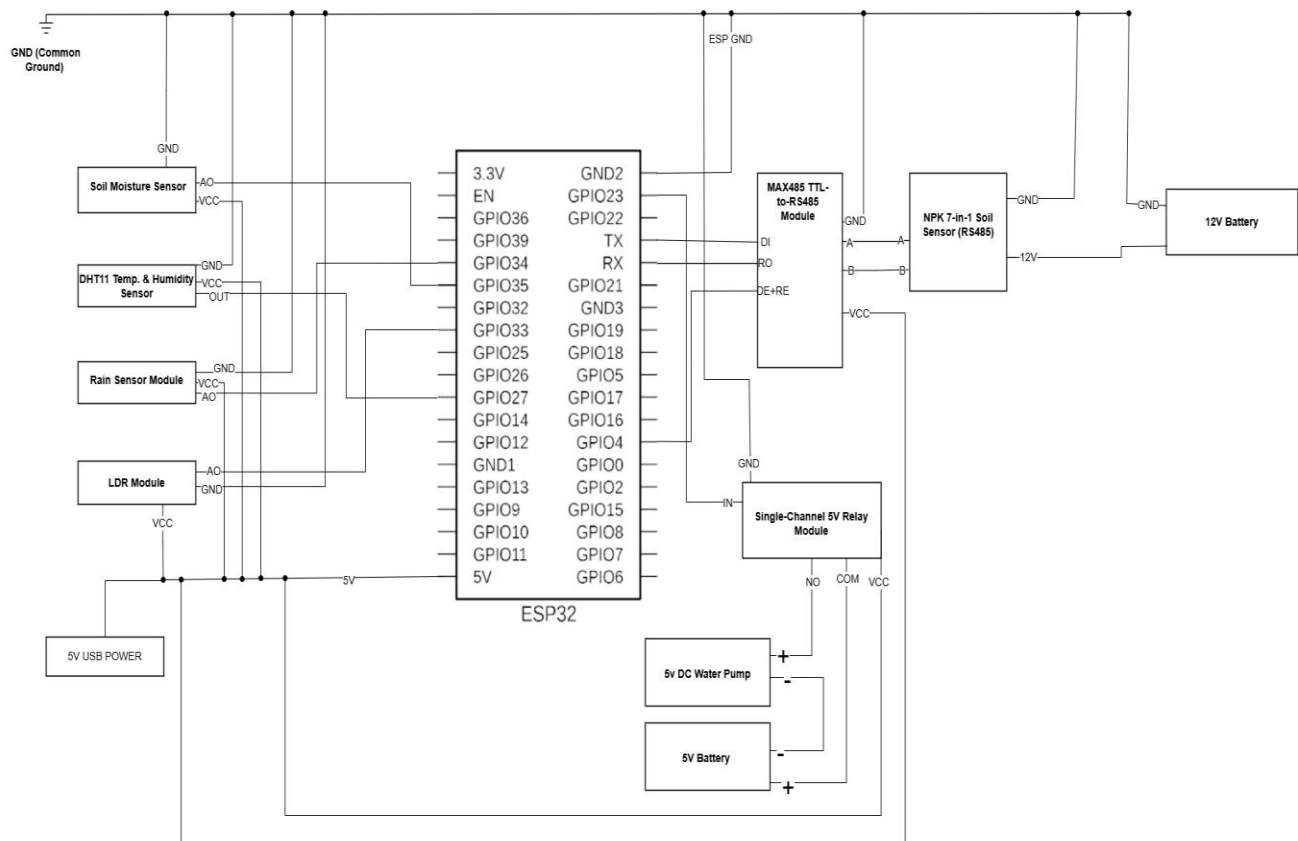


Figure 4.9 Hardware Schematic Diagram

4.10 Timing Diagram

Figure 4.10 can be used to visualize the sequence of events that happened over time when monitoring and irrigation control was done in real time with the SmartCrop system. It highlights the timing relationship between the ESP32 microcontroller, the Firebase cloud services, the mobile app and the relay module and irrigation pump.

In the first phase, the ESP32 reads the sensor values on a regular interval, as shown, and uploads the information to Firebase. After synchronization, the values are downloaded to the farmer's dashboard in the Flutter application. When the farmer uses the mobile app to send the command, the command is sent to Firebase, and is now accessible to the ESP32. The ESP32 uses a continuously polling method to check the irrigation status, and is able to send the ON command to the relay module and start the water pump when it receives the ON command. Likewise, an OFF command will cause the relay to shut off and irrigation to stop.

The timing diagram illustrates the synchronization between the various components of the system, including the sensors, the cloud communication interface, the users, and the actuators, ensuring that all the components of the system are synchronized with each other in time.

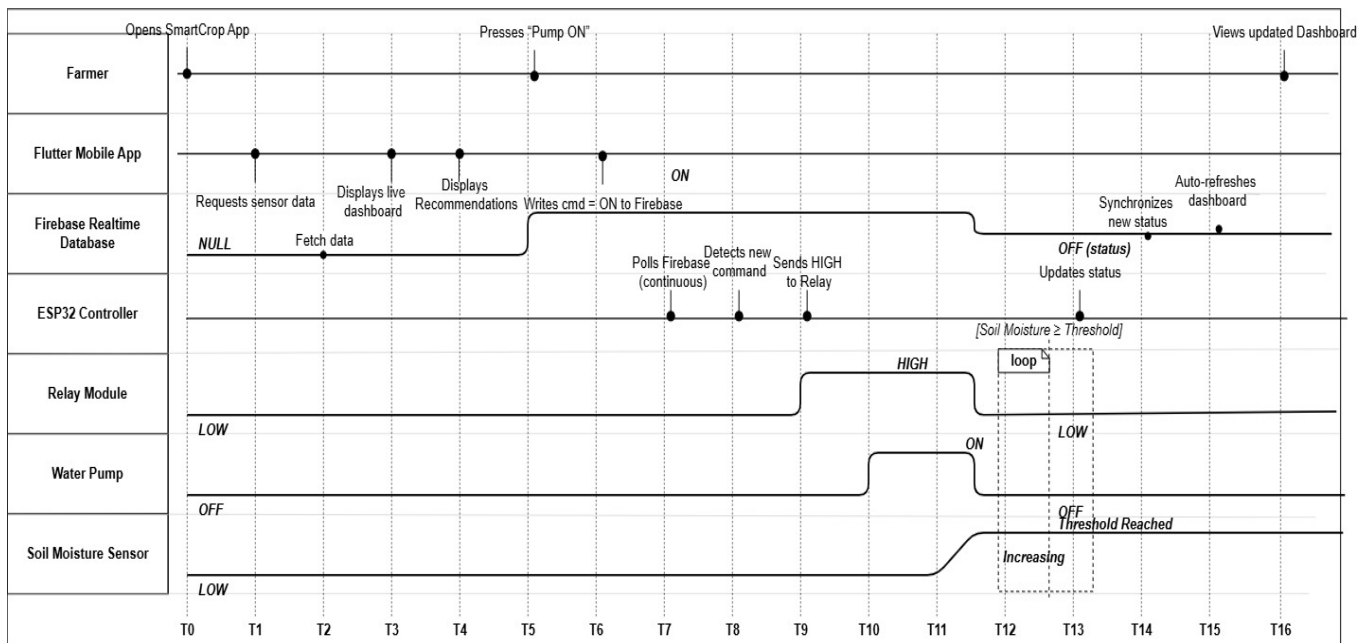


Figure 4.10 Timing Diagram

5. Data Design

SmartCrop system uses Firebase Realtime Database as a cloud-based NoSQL data model to store and synchronise the application data. Firebase stores data in a hierarchical JSON structure, which allows real-time communication between the ESP32 microcontroller, a Flutter-based mobile app, and the cloud services.

The data design aims to enable efficient storage, retrieval, and synchronization of user data, sensor readings, machine learning results, weather information, irrigation status, and historical data. A unique User ID (UID) is generated by Firebase Authentication, used to identify each registered farmer. This UID provides a primary identifier to be used to link user-specific information with agriculture data.

The database is structured into individual logical nodes, corresponding to individual system modules. The data generated by the values from the sensors at the ESP32 are updated on a continuous basis in the SensorData node, while output of the machine learning models is stored in the nodes dedicated to crop recommendations, yield predictions and fertilizer recommendations. Weather data received from the Weather API is kept in a separate table to be used for irrigation planning. The irrigation module holds the irrigation status information and the relay module is connected to the water pump that can be remotely controlled.

The data collected from the sensors and the recommendations generated are stored for long-term monitoring and future analysis. The modular design of the database also enables scalability, as more sensors, machine learning models, and system capabilities can be added to the database without significant changes to its structure.

Integrating the data structure with the overall application design improves the efficiency of data synchronization, reduces data duplication, facilitates real-time updates, and ensures scalability for future developments.

5.1 Data Dictionary

The following data dictionary specifies the main entities utilized in the SmartCrop system.

Table 5.1 describes the information stored for each registered user in the SmartCrop system. It includes user identification, personal details, farm location, and the account creation timestamp.

Attribute	Data Type	Description
uid	String	Unique identifier generated by Firebase Authentication
fullName	String	Farmer's full name
email	String	Registered email address
phone	String	Contact number
farmLocation	String	Farm location
createdAt	Timestamp	Account creation date

Table 5.1 User

Table 5.2 stores real-time sensor readings collected from the ESP32-based IoT devices. It contains soil and environmental parameters along with the timestamp indicating when the data was recorded.

Attribute	Data Type	Description
moisture	Float	Soil moisture percentage
temperature	Float	Ambient temperature (°C)
humidity	Float	Relative humidity (%)
pH	Float	Soil pH value
nitrogen	Float	Nitrogen concentration
phosphorus	Float	Phosphorus concentration
potassium	Float	Potassium concentration
ec	Float	Electrical Conductivity
timestamp	Timestamp	Date and time of data collection

Table 5.2 SensorData

Table 5.3 contains the crop recommendations generated by the machine learning model. It stores the recommended crop, prediction confidence, and the date on which the recommendation was generated.

Attribute	Data Type	Description
cropName	String	Recommended crop
confidence	Float	Prediction confidence (%)
recommendationDate	Timestamp	Recommendation generation date

Table 5.3 CropRecommendation

Table 5.4 stores the predicted crop yield generated by the AI model. It includes the selected crop, estimated yield, measurement unit, and prediction date.

Attribute	Data Type	Description
cropName	String	Selected crop
predictedYield	Float	Estimated yield
unit	String	kg/acre
predictionDate	Timestamp	Prediction date

Table 5.4 YieldPrediction

Table 5.5 contains fertilizer recommendations based on soil conditions. It records the recommended fertilizer, required quantity, measurement unit, and recommendation date.

Attribute	Data Type	Description
fertilizerName	String	Recommended fertilizer
quantity	Float	Recommended quantity
unit	String	kg/acre
recommendationDate	Timestamp	Recommendation generation date

Table 5.5 FertilizerRecommendation

Table 5.6 stores weather forecast information retrieved from the Weather API. It includes forecasted temperature, humidity, rainfall, weather conditions, and forecast date.

Attribute	Data Type	Description
temperature	Float	Forecast temperature
humidity	Float	Forecast humidity
rainfall	Float	Expected rainfall
weatherCondition	String	Weather description
forecastDate	Date	Forecast date

Table 5.6 Weather

Table 5.7 stores the irrigation control status of the system. It records whether the water pump is ON or OFF, the last update time, and the user who performed the action.

Attribute	Data Type	Description
status	Boolean	ON/OFF state
lastUpdated	Timestamp	Last status update
controlledBy	String	User initiating the action

Table 5.7 Irrigation

Table 5.8 maintains historical records of sensor readings and AI-generated recommendations. These records are stored with timestamps to support monitoring, analysis, and future reference.

Attribute	Data Type	Description
sensorData	Object	Previous sensor readings
recommendations	Object	Previous AI recommendations
timestamp	Timestamp	Record creation time

Table 5.8 HistoricalData

6. Algorithm & Implementation

In this section, the major algorithms to be implemented inside the SmartCrop system are described. The algorithms are shown in pseudo code so that the actions of each program module are explained without the need to specify a language. The integration features Flutter, Firebase, firmware based on the ESP32, machine learning models, and external APIs, all working together to deliver intelligent agricultural services.

6.1 Algorithm 1 – User Authentication

Purpose: Authenticate users using Firebase Authentication.

Start

Receive email and password

Validate input fields

IF credentials are valid THEN

 Authenticate user using Firebase

 IF authentication successful THEN

 Open Dashboard

 ELSE

 Display login error

ENDIF

End

6.2 Algorithm 2 – Sensor Data Acquisition

Purpose: Read sensor values using ESP32.

Start

Initialize sensors

Read Soil Moisture

Read Temperature

Read Humidity

Read pH

Read NPK values

Read EC

Upload values to Firebase

Wait predefined interval

Repeat

6.3 Algorithm 3 – Crop Recommendation

Purpose: Recommend the most suitable crop.

Start

Retrieve latest sensor values

Preprocess data

Input data into ML model

Generate prediction

Store recommendation in Firebase

Display recommended crop

End

6.4 Algorithm 4 – Yield Prediction

Purpose: Estimate crop production.

Start

Retrieve recommended crop

Retrieve latest sensor values

Input data into Yield Prediction Model

Calculate estimated yield

Store result

Display predicted yield

End

6.5 Algorithm 5 – Fertilizer Recommendation

Purpose: Recommend fertilizer according to soil nutrients.

Start

Retrieve NPK values

Retrieve pH value

Analyze nutrient deficiency

Generate fertilizer recommendation

Store recommendation

Display result

End

6.6 Algorithm 6 – Weather Retrieval

Purpose: Retrieve weather forecast.

Start

Connect to Weather API

Request forecast

Receive response

Extract rainfall prediction

Display weather information

End

6.7 Algorithm 7 – Irrigation Control

Purpose: Operate irrigation remotely.

Start

User selects ON/OFF

Update Firebase

ESP32 monitors irrigation status

IF status = ON

 Activate relay

ELSE

 Deactivate relay

ENDIF

End

7. Software Requirements Traceability Matrix

Table 7.1 shows how each of the functional requirements listed in the Software Requirements Specification (SRS) has been satisfied in the software design. It creates a traceability link between the system requirements and the design elements, so that all of the requirements are included in the overall design.

Req. No.	Requirement	Ref. Item	Design Component	Component Items
FR-01	User Registration	Class Diagram	User	registerUser()
		Activity Diagram	Registration Process	User Registration
FR-02	User Login	Class Diagram	Authentication	login()
		Sequence Diagram	Login Process	User Authentication
FR-03	View Dashboard	Activity Diagram	Dashboard	Dashboard Flow
FR-04	Monitor Live Sensor Data	DFD Level 1	Sensor Monitoring	Read Sensor Data
		Sequence Diagram	Sensor Monitoring	ESP32 → Firebase → App
FR-05	View Historical Data	DFD Level 1	History Module	Retrieve Historical Data

Req. No.	Requirement	Ref. Item	Design Component	Component Items
FR-06	Crop Recommendation	Sequence Diagram	AI Module	Predict Crop
FR-07	Yield Prediction	Sequence Diagram	AI Module	Predict Yield
FR-08	Fertilizer Recommendation	Sequence Diagram	AI Module	Recommend Fertilizer
FR-09	Weather Forecast	Sequence Diagram	Weather Module	Fetch Forecast
FR-10	Control Irrigation	Timing Diagram	Irrigation Module	Relay Control
		DFD Level 1	Irrigation Process	Update Pump Status
FR-11	Manage Profile	Class Diagram	User	updateProfile()
FR-12	Logout	Activity Diagram	Authentication	Logout Process

Table 7.1 Software Requirements Traceability Matrix

8. Human Interface Design

The SmartCrop mobile application will be built to give a simple, intuitive and user friendly interface for farmers to observe agricultural conditions and run farming activities efficiently. The application is designed to adhere to Material Design guidelines according to the Flutter framework, which guarantees consistency, response and ease of use on Android devices.

The user interface features several screens with their respective functions, including authentication, a dashboard for monitoring systems, the recommendation by the AI, weather forecast, irrigation control, historical data visualization, and profile management. All the major features are easy to reach via the main dashboard, reducing user effort with the navigation structure.

Throughout the application, input validation is included, navigation controls are clear, icons are meaningful, and there is immediate visual feedback on the user's actions, which boosts usability and minimizes errors. Immediate synchronization with Firebase means that when the sensor values are updated and the irrigation status is changed, these updates are immediately updated on the user interface.

Each of the next sections provides the essential screens of the SmartCrop application, along with what each screen is used for and what kind of actions can be taken with the application.

8.1 Screen Images

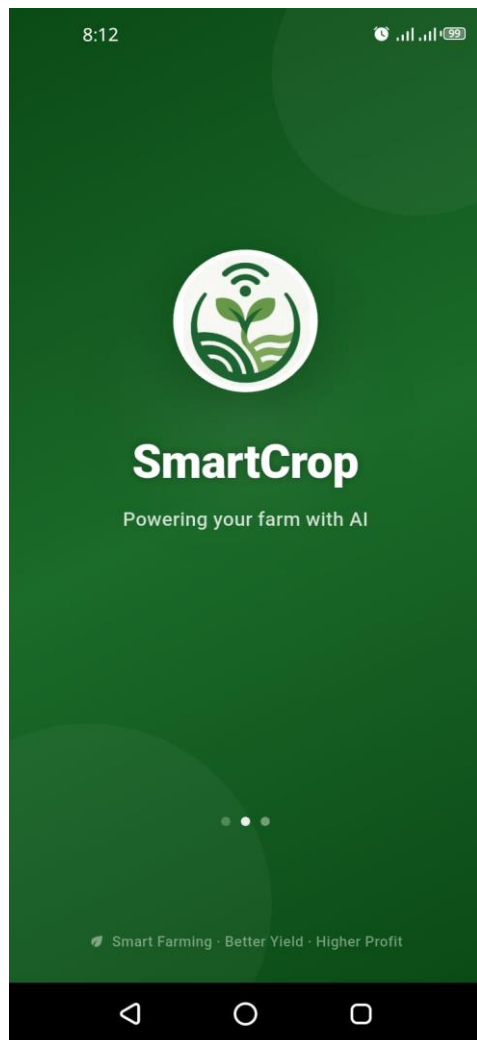


Figure 8.1 Splash Screen

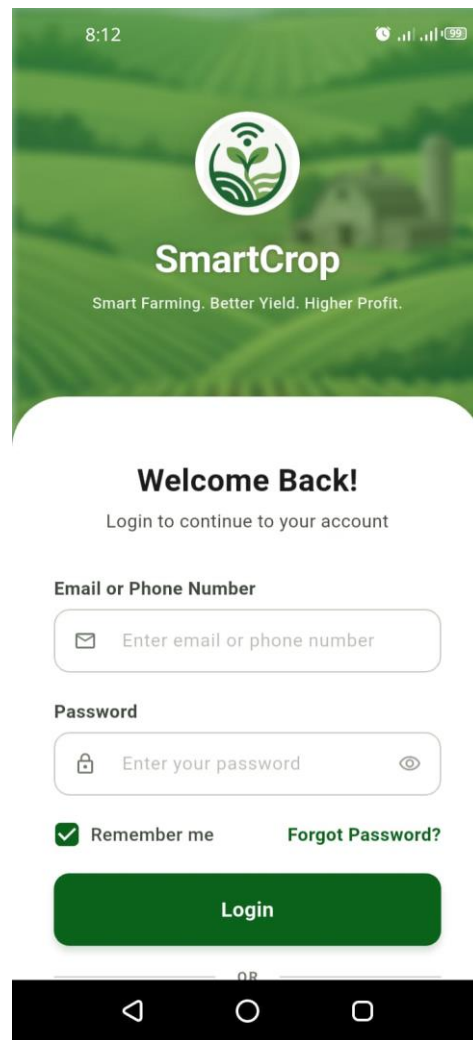


Figure 8.2 Login Screen

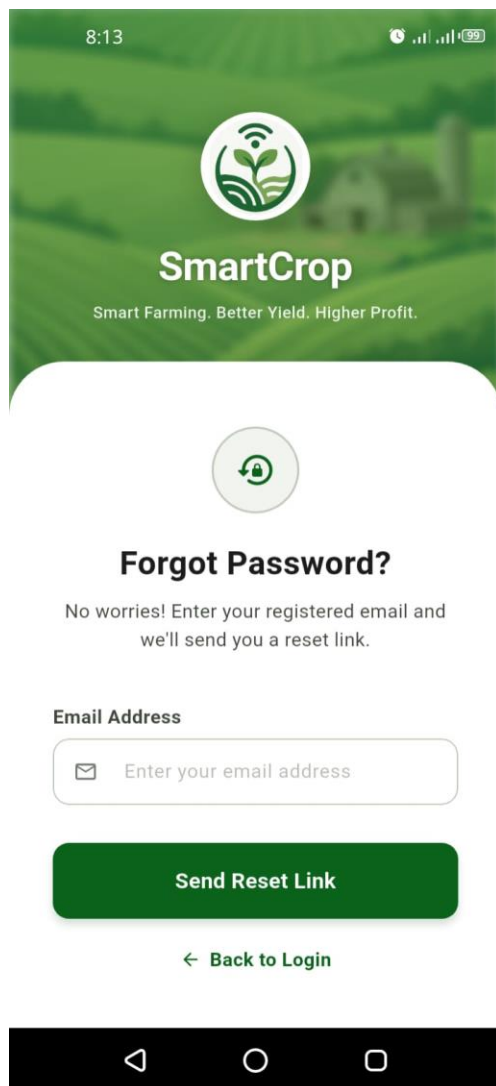


Figure 8.3 Forgot Password Screen

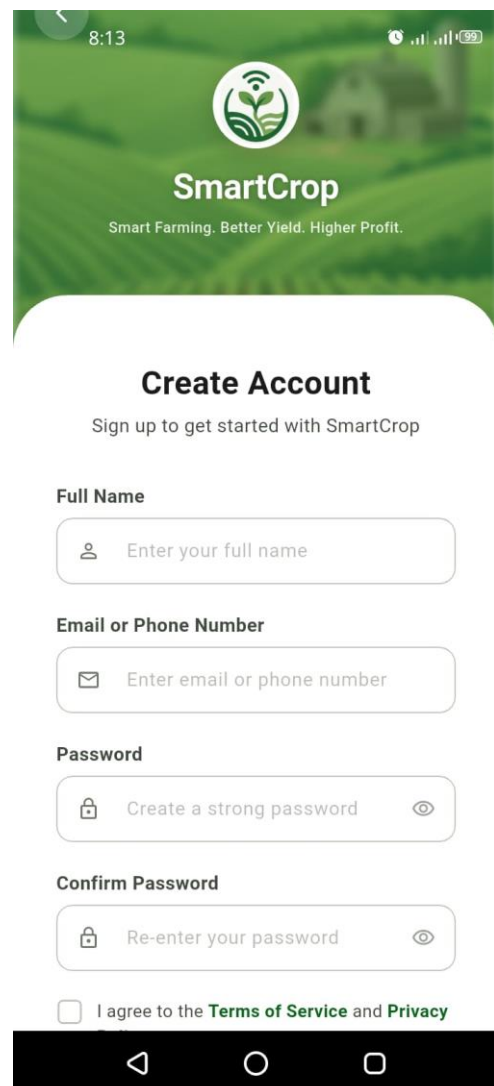


Figure 8.4 Sign Up (Registration) Screen

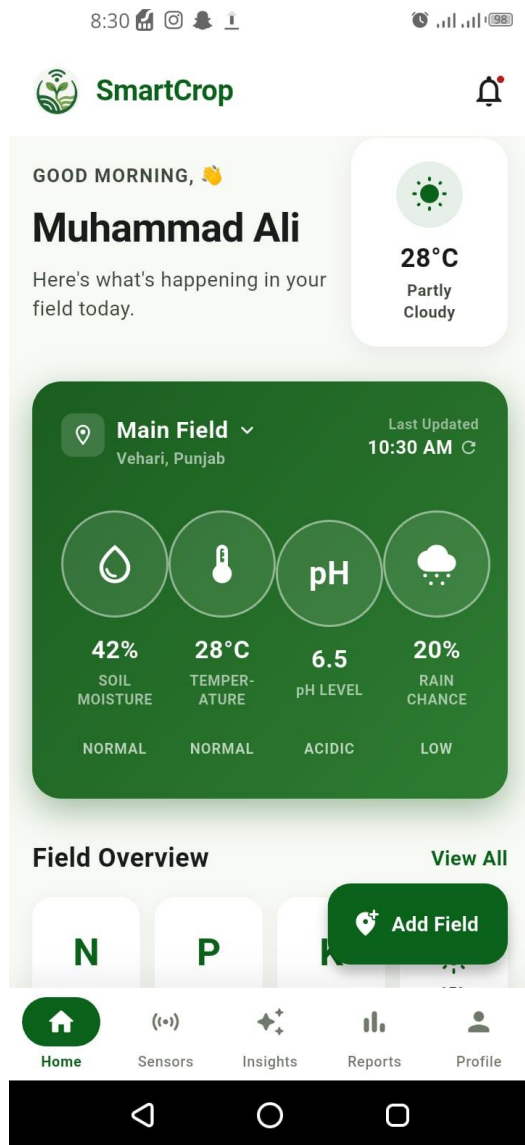


Figure 8.5 Dashboard

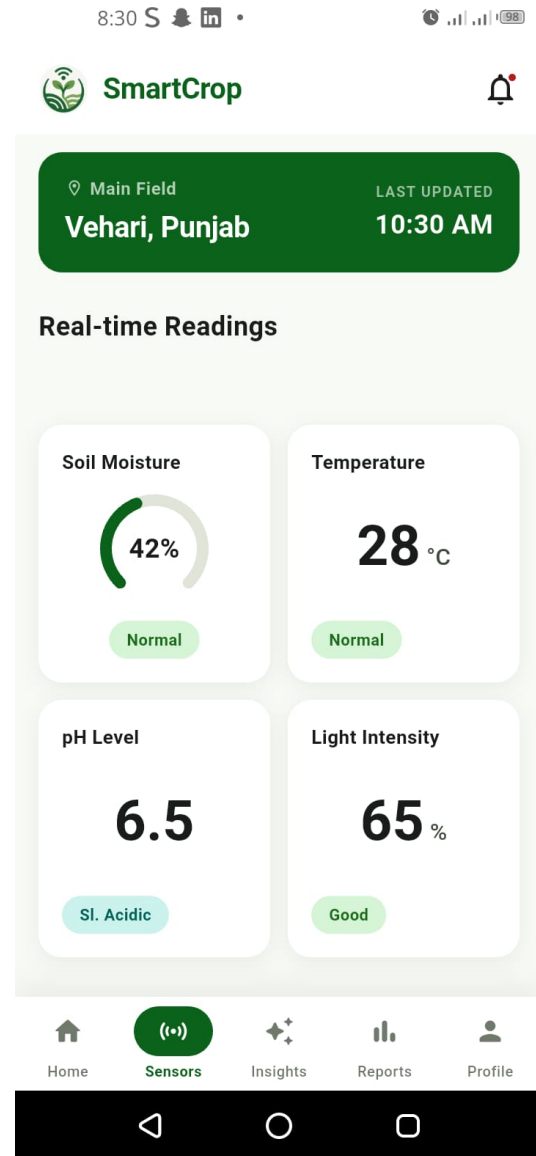


Figure 8.6 Live Sensor Monitoring

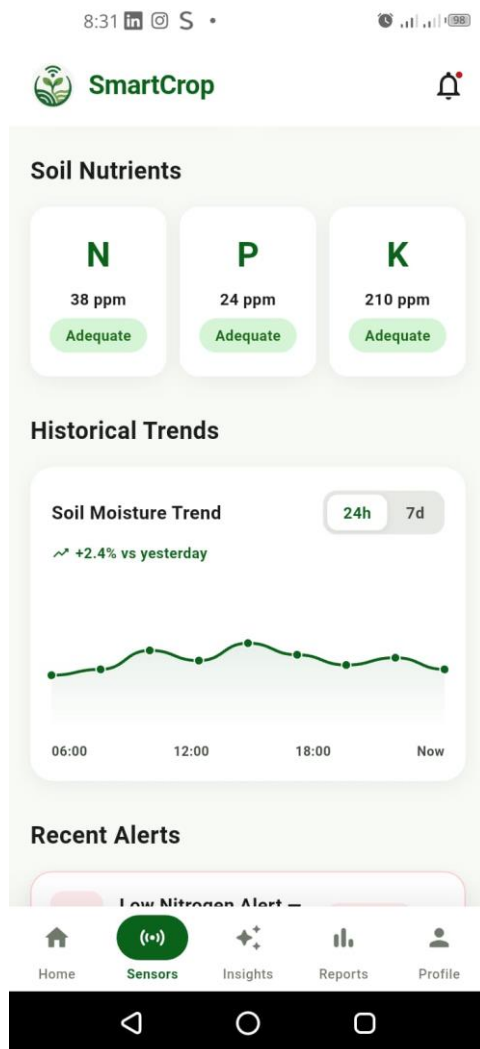


Figure 8.7 Historical trends

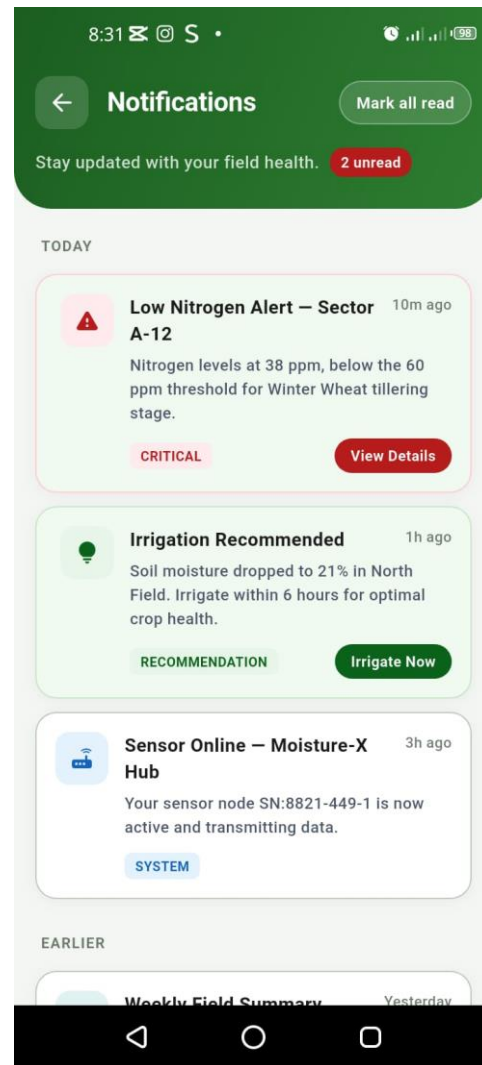


Figure 8.8 Notification Screen

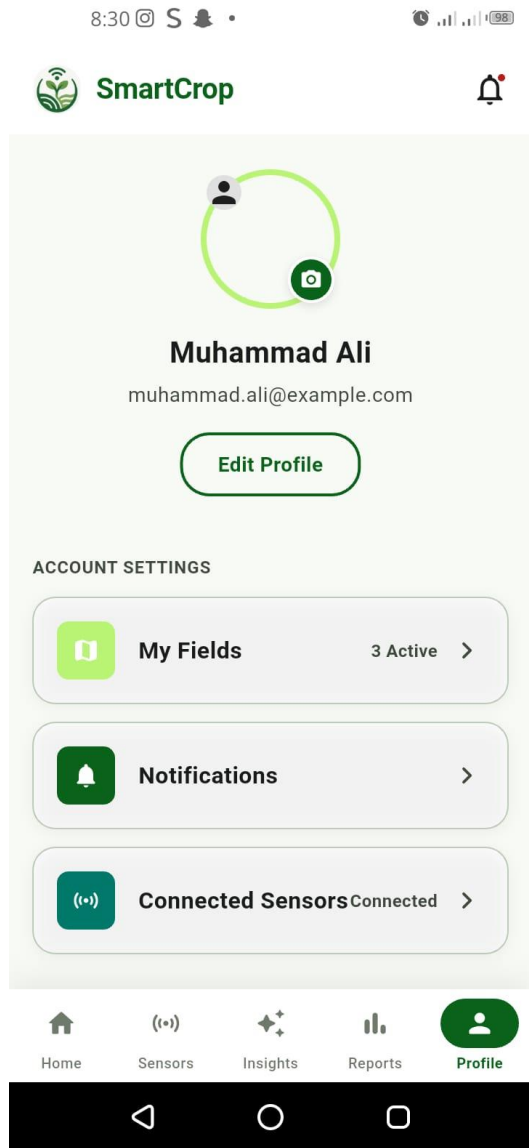


Figure 8.9 Profile Screen

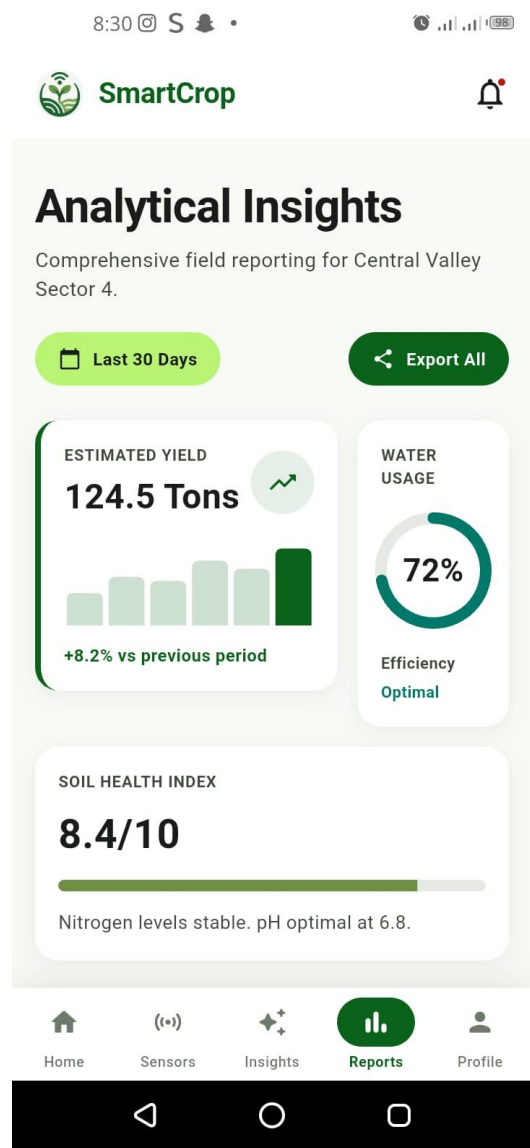


Figure 8.10 Analytical Insights

8.2 Screen Objects and Actions

Table 8.1 summarizes the primary interface components available within the SmartCrop application and the actions performed by each component.

Screen	Screen Object	Action
Login	Email Text Field	Enter registered email
Login	Password Field	Enter password
Login	Login Button	Authenticate user
Login	Register Link	Open registration screen
Registration	Input Fields	Enter user information
Registration	Register Button	Create new account
Dashboard	Navigation Cards	Open selected module
Dashboard	Refresh Button	Update latest sensor values
Live Monitoring	Sensor Cards	Display real-time sensor readings
Crop Recommendation	Recommendation Card	Display suggested crop
Yield Prediction	Prediction Card	Display predicted yield
Fertilizer Recommendation	Recommendation Card	Display fertilizer details
Weather Forecast	Weather Panel	Display forecast information
Irrigation	ON Button	Start irrigation
Irrigation	OFF Button	Stop irrigation
Historical Data	History List	View previous records
Profile	Edit Button	Update profile
Profile	Logout Button	End user session

Table 8.1 Screen Objects and Actions

9. References

- [1] Flutter, "Flutter Documentation," Google, 2026. [Online]. Available: <https://docs.flutter.dev>. [Accessed: Jul. 10, 2026].
- [2] Google Firebase, "Firebase Documentation," Google, 2026. [Online]. Available: <https://firebase.google.com/docs>. [Accessed: Jul. 10, 2026].
- [3] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, Shanghai, China, 2025.
- [4] OpenWeather, "Weather API Documentation," 2026. [Online]. Available: <https://openweathermap.org/api>. [Accessed: Jul. 10, 2026].
- [5] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [6] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [7] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [9] T. Erl, *Cloud Computing: Concepts, Technology & Architecture*. Upper Saddle River, NJ, USA: Prentice Hall, 2013.
- [10] M. A. Razzaque, M. Milojevic-Jevric, A. Palade, and S. Clarke, "Middleware for Internet of Things: A Survey," *IEEE Internet of Things Journal*, vol. 3, no. 1, pp. 70–95, Feb. 2016.